Lecture 23: Variational Autoencoders

Give the latent space no holes, then sample it

Parts of this chapter adapt LMU I2DL (slds-lmu), CC BY 4.0.

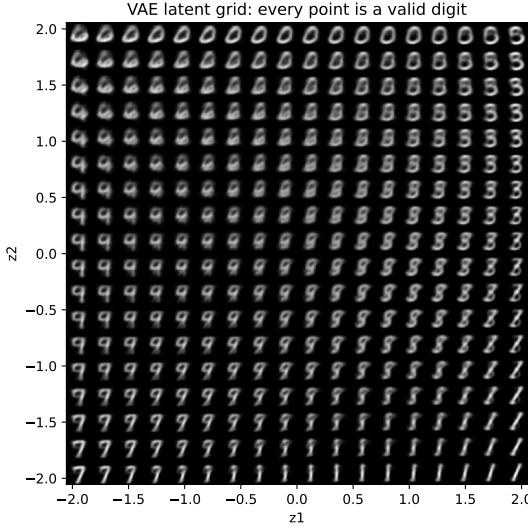
What if every point were a digit?

Last lecture ended in failure: the plain autoencoder could not **generate** - random codes decode to blurry blobs.

But look what we *want* (right): a latent space where **every point decodes to a valid digit**, and neighbours morph smoothly into one another.

If the whole space were like this, a **random draw** would produce a brand-new digit. That is a **generative model**. This lecture builds it.

VAEs come from **Kingma & Welling (2013)**, "Auto-Encoding Variational Bayes" - the same Kingma who gave you the **Adam** optimizer.



Outline

From autoencoder to generative model

The VAE objective: the ELBO

The reparameterization trick

The VAE in action

Honest limits and the road ahead

From autoencoder to generative model

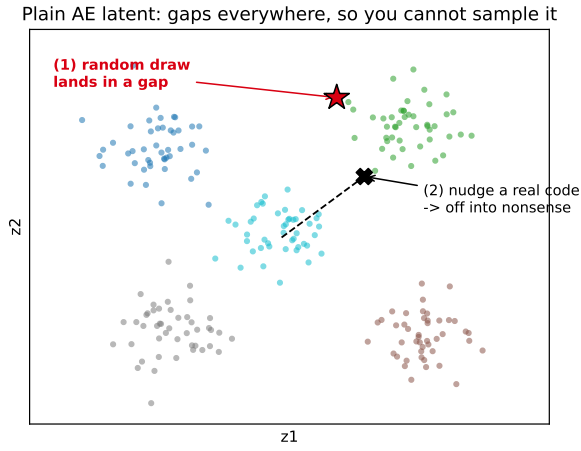
An autoencoder memorizes points. We want a space we can sample.

Why the plain autoencoder can't generate

A plain autoencoder maps each input to a single **point z**, and never organizes those points. So generating fails two ways (right):

1. a **random draw** lands in a *gap* where no code lives → the decoder makes garbage;
2. even **nudging a real code** drifts into a gap → nonsense: nearby points were never made to mean similar things.

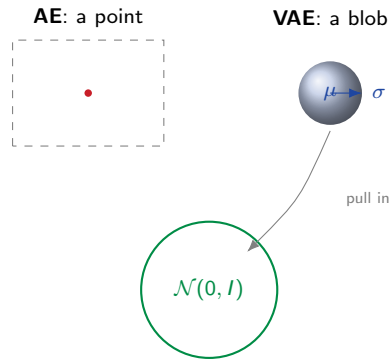
The latent is full of holes. You can **reconstruct**, but you cannot **sample**.



The fix: a blob, not a point

The fix, in one line: make the encoder output a little **Gaussian blob** per input - a mean $\boldsymbol{\mu}(\mathbf{x})$ and a spread $\boldsymbol{\sigma}(\mathbf{x})$ - and push all the blobs to fill a standard normal $\mathcal{N}(0, I)$ with no gaps.

Two new ingredients: **(1)** the encoder predicts $\boldsymbol{\mu}, \boldsymbol{\sigma}$; **(2)** a regularizer pulls every blob toward $\mathcal{N}(0, I)$. The next section makes each precise.



The VAE objective: the ELBO

Where the loss comes from - honestly.

First, what are we optimizing?

A **generative** model is one that thinks real digits are **likely** and everything else unlikely. If it does, then a sample drawn from it looks like a real digit.

So the goal is concrete. Write $p(\mathbf{x})$ for the probability the model gives an image $\mathbf{x}$; we want the training images to score high:

$$\text{maximize} \sum_{\text{training images}} \log p(\mathbf{x}).$$

This is just **maximum likelihood** - the same "make the data likely" you have used all course. The only twist: here $p(\mathbf{x})$ is defined through a hidden latent code (next slide).

Where does $p(\mathbf{x})$ come from?

We define the model by a simple **two-step recipe** for making an image:

1. draw a code from a fixed, simple **prior** $p(\mathbf{z}) = \mathcal{N}(0, I)$ - "pick a random point near the origin, from a bell curve";
2. push it through the **decoder** $p(\mathbf{x} \mid \mathbf{z})$ (the neural net) to get an image.

The probability of an image is then the chance that *any* code produces it - add up over all codes:

$$p(\mathbf{x}) = \int p(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}.$$

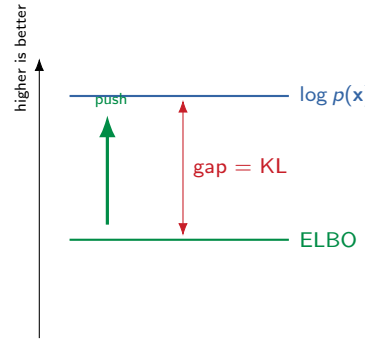
That "add up over *all* codes" is the trouble: the integral is **intractable** - we cannot compute it, let alone maximize it. We need a shortcut.

The plan: push up a lower bound

We cannot maximize $\log p(\mathbf{x})$ directly. **The trick:** build a quantity we *can* compute, the **ELBO** (Evidence Lower Bound), that is always $\leq \log p(\mathbf{x})$. Push the ELBO up, and $\log p(\mathbf{x})$ is dragged up with it.

To build it we bring in the **encoder** $q(\mathbf{z} \mid \mathbf{x})$: given an image, its best guess at which codes could have made it.

The **gap** between floor (ELBO) and ceiling ($\log p(\mathbf{x})$) is exactly how wrong that guess is - so raising the ELBO does two jobs: fit the data better *and* sharpen the encoder. ($\mathrm{KL}(a\|b)$ below just measures how far apart two distributions are, zero when they match.)



Its loss is a **tug of war**: reconstruction spreads codes apart, the KL tidies them toward $\mathcal{N}(0, I)$.

Deriving the ELBO

Start from $\log p(\mathbf{x})$, multiply inside by $\frac{q(\mathbf{z}|\mathbf{x})}{q(\mathbf{z}|\mathbf{x})}$, and split the logarithm:

$$\log p(\mathbf{x}) = \underbrace{\mathbb{E}_{q(\mathbf{z}|\mathbf{x})}\big[\log p(\mathbf{x} \mid \mathbf{z})\big] - \mathrm{KL}\big(q(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z})\big)}_{\mathrm{ELBO}(\mathbf{x})} + \underbrace{\mathrm{KL}\big(q(\mathbf{z} \mid \mathbf{x}) \,\|\, p(\mathbf{z} \mid \mathbf{x})\big)}_{\geq\, 0}.$$

The last term is a KL divergence, so it is ≥ 0. Drop it and you get a **lower bound**:

$$\log p(\mathbf{x}) \;\geq\; \mathrm{ELBO}(\mathbf{x}).$$

We cannot maximize $\log p(\mathbf{x})$, so we **maximize the ELBO** instead.

Reading the two terms

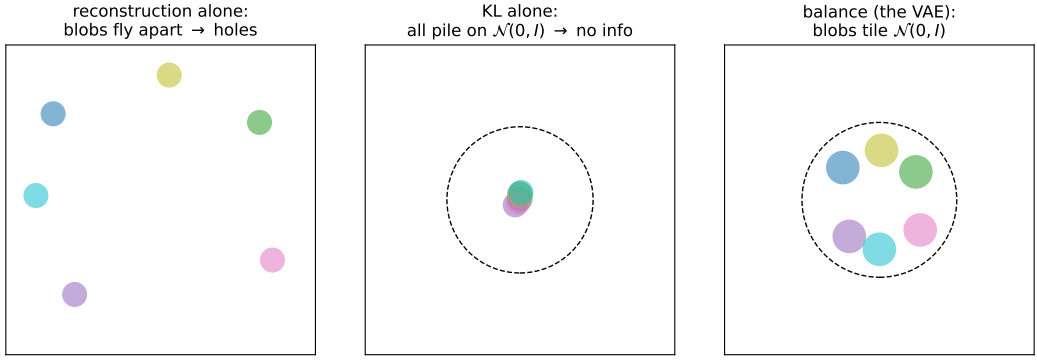
$$\text{ELBO}(\mathbf{x}) = \underbrace{\mathbb{E}_{q(\mathbf{z}|\mathbf{x})}\big[\log p(\mathbf{x} \mid \mathbf{z})\big]}_{\text{reconstruction}} - \underbrace{\mathrm{KL}\big(q(\mathbf{z} \mid \mathbf{x}) \,\|\, \mathcal{N}(0, I)\big)}_{\text{regularizer}}$$

- **Reconstruction:** decode $\mathbf{z}$ and rebuild $\mathbf{x}$ well - this is exactly the autoencoder loss from L22.
- **KL regularizer:** keep each blob $q(\mathbf{z} \mid \mathbf{x})$ close to $\mathcal{N}(0, I)$ - this is what fills the latent space with no holes.

Callback: the KL term is a **prior** on the code - a regularizer, the same role weight decay played in ch5. Closed form for two Gaussians: $\mathrm{KL} = \frac{1}{2}\sum_j \left(\mu_j^2 + \sigma_j^2 - \ln \sigma_j^2 - 1\right)$, smallest when $\mu = 0, \sigma = 1$ - i.e. the blob *is* the prior.

The tug of war

The two ELBO terms pull in opposite directions - their **balance** is what makes the latent both faithful and samplable:



Reconstruction alone spreads codes apart (sharp rebuilds, but the holes come back); the **KL** alone collapses them onto the prior (samplable, but they carry no information). The VAE keeps just enough of each.

The reparameterization trick

You cannot differentiate a coin flip. Unless.

Backprop through randomness

The ELBO needs a sample $\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$. But **sampling is random** - there is no gradient through a random node, so backprop stops dead.

Predict: we need a derivative through a random draw. How?

Backprop through randomness

The ELBO needs a sample $\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$. But **sampling is random** - there is no gradient through a random node, so backprop stops dead.

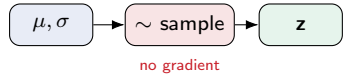
Predict: we need a derivative through a random draw. How?

The trick: move the randomness into an *input*.

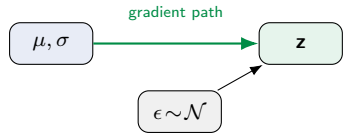
$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(0, I).$$

Now $\boldsymbol{\mu}, \boldsymbol{\sigma}$ are deterministic - gradients flow through them; only $\boldsymbol{\epsilon}$ is random, and it carries no parameters.

before: gradient blocked



after: gradient flows



With this, the VAE trains end to end with plain SGD / Adam - the same optimizer from ch5.

The VAE in action

Sample the prior, decode, done.

Generate: sample the prior, decode

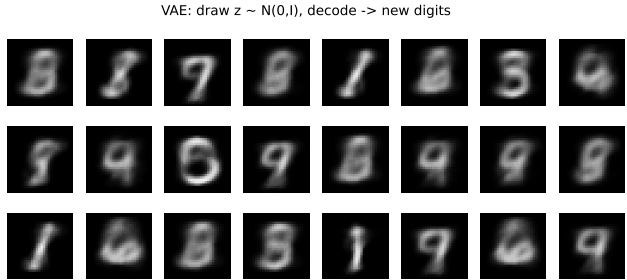
Training is done. To generate a brand-new digit that was never in the data:

1. draw $\mathbf{z} \sim \mathcal{N}(0, I)$,
2. decode $\hat{\mathbf{x}} = g(\mathbf{z})$.

That is it. Because the KL term made the codes fill $\mathcal{N}(0, I)$, almost every draw decodes to a valid digit.

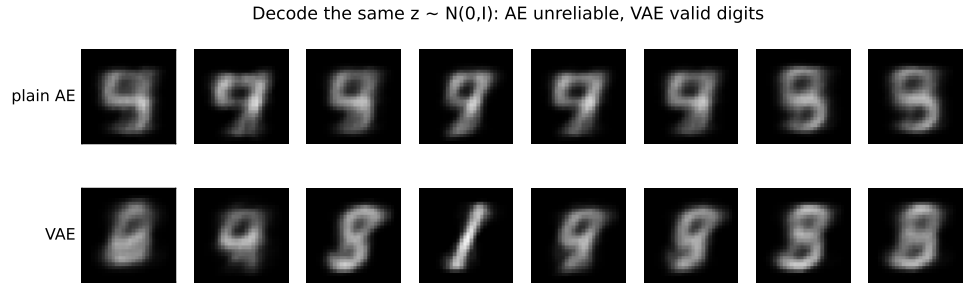
This is the step the plain autoencoder could not take - its codes never matched any distribution you could sample.

Not only digits: the same recipe generates **molecules** (drug discovery), **text**, **tabular** rows, **audio** and **time series** - anything you can decode.



The fix, side by side

Decode the *same* $\mathcal{N}(0, I)$ draws with each model:

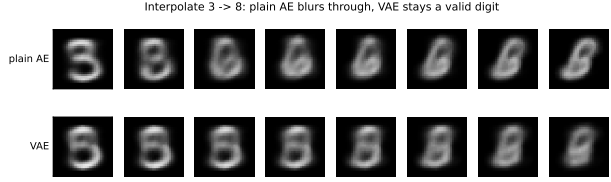


The plain AE collapses to repetitive, unreliable blobs (its codes do not live where you are sampling). The VAE gives **diverse, valid digits** - the KL prior did its job.

Walk the latent space

Because neighbouring codes decode to similar digits, you can **interpolate**: encode a 3 and an 8, walk the straight line between their codes, decode each step.

The VAE path stays closer to valid digits than the plain AE's (top row). And the full latent grid from the cold open shows the same thing everywhere: a smooth, organized space you can **traverse**.



One knob: β-VAE and disentanglement

Multiply the KL term by a weight β:

$$\mathcal{L} = \underbrace{\mathbb{E}_q[\log p(\mathbf{x} \mid \mathbf{z})]}_{\text{reconstruction}} - \beta \cdot \mathrm{KL}\big(q(\mathbf{z} \mid \mathbf{x}) \,\|\, \mathcal{N}(0, I)\big).$$

- β large: press harder toward $\mathcal{N}(0, I)$. The latent axes tend to **disentangle** into interpretable factors - one axis becomes rotation, another thickness, another width.
- The price: blurrier reconstructions (you spent capacity on a tidy latent).

It is one of a family of ELBO variants: **conditional** VAE (generate a chosen class), **VQ**-VAE (a discrete latent, sharper), and more - each a small twist on the same objective.

The VAE is inside modern image generators

You might think diffusion models replaced the VAE. The opposite is true - they run *inside* one.

- **Latent diffusion (Stable Diffusion).** Do not diffuse pixels - diffuse in a **VAE's latent**. The VAE compresses a $512 \times 512 \times 3$ image to $64 \times 64 \times 4$ (about $48\times$ fewer numbers); the diffusion model works in that latent; the **VAE decoder** paints the final image. Every Stable Diffusion or Midjourney image is decoded by a VAE.
- **Images as tokens (VQ-VAE).** Give the VAE a discrete codebook and an image becomes a **sequence of tokens** - so a transformer can generate images the same way an LLM generates text (the DALL-E lineage).

The VAE did not lose to diffusion. It became the **compression layer inside it**.

Honest limits and the road ahead

Principled, but blurry.

VAE samples are blurry

You have seen it on every VAE figure in this deck: the samples are recognizable but **soft**. It gets worse on bigger images - a VAE on **CelebA** faces gives a recognizable face but a **blurry-mess background**.

Why: the reconstruction loss rewards the **average** of all plausible outputs, and a Gaussian decoder smears detail. A VAE gives you a **principled latent space and a likelihood**, but not sharp images.

VAE	principled latent + likelihood; blurry
GAN	sharp images; no likelihood, unstable (*L19 showcase*)
Diffusion	state of the art - and it runs *inside* a VAE latent

Recap and the road ahead

- A VAE is an autoencoder whose encoder outputs a **distribution** per input, trained to fill $\mathcal{N}(0, I)$.
- Its loss is the **ELBO** = reconstruction − KL; the KL is a prior that removes the holes.
- The **reparameterization trick** makes the random sampling differentiable, so it trains with plain gradient descent.
- Sample the prior to **generate**; samples are valid but blurry.

Next - two chapters, one from each thread. The **attention** thread from [54] continues in the **attention chapter** (L24+): transformers and LLMs. This **generative** thread continues in the **diffusion chapter** (L27-L31), which picks up exactly where these blurry samples leave off.